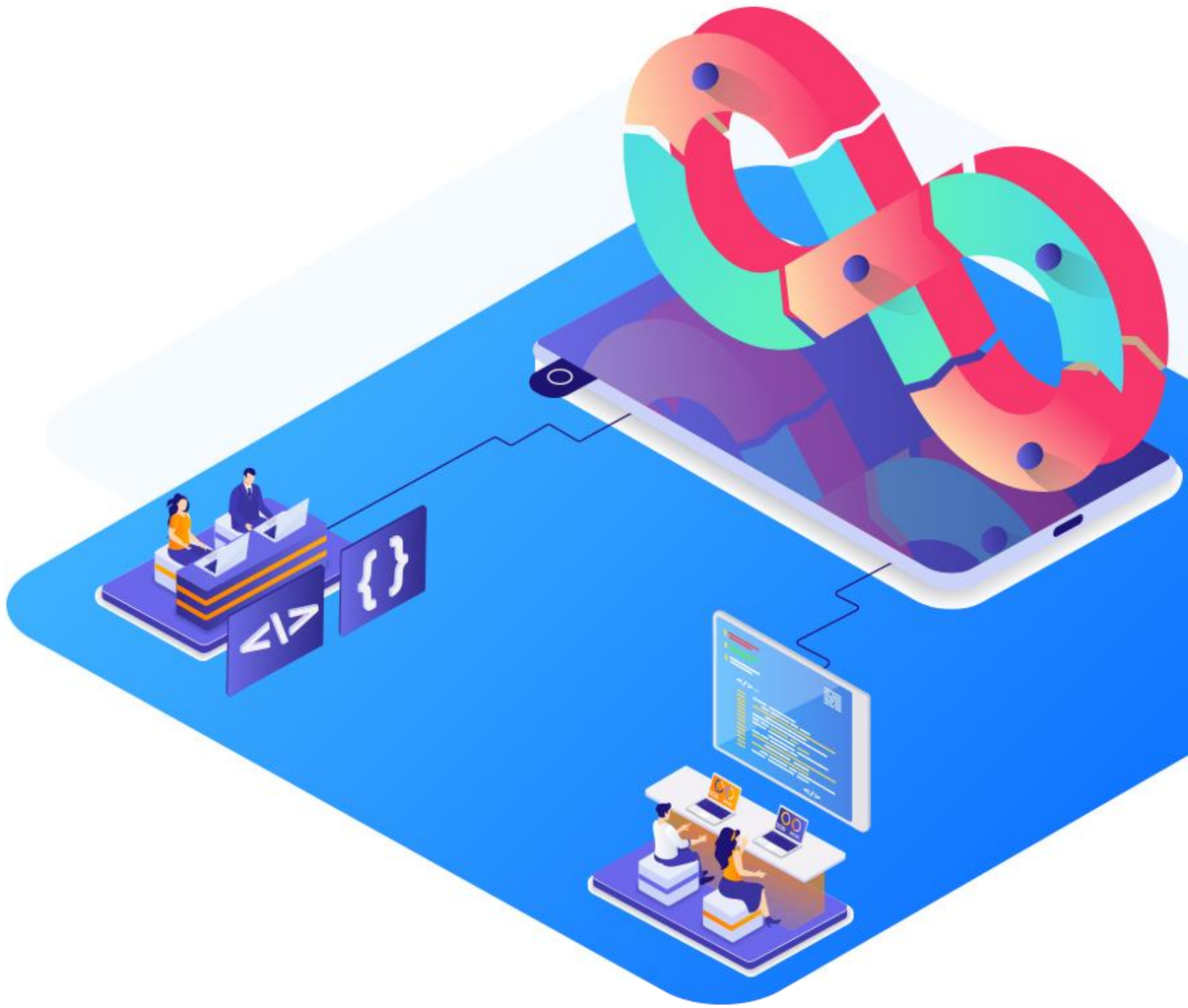
DevOps on AWS



Course-End Project

Automating CI/CD Pipeline for Spring Boot Application Deployment on AWS

Objective

To implement a CI/CD pipeline using AWS services for automating the deployment of a Spring Boot application on Amazon ECS with Docker, integrating CodePipeline, CodeBuild and ECR for seamless updates

COURSE-END PROJECT

Problem Statement and Motivation

Real-time scenario:

Imagine a retail company that has a Spring Boot-based inventory management system. The system is used by several departments and is hosted in containers to ensure scalability.

The company wants to reduce the manual intervention required to deploy updates by automating the build and deployment process. By setting up a CI/CD pipeline, developers can push updates to a GitHub repository, automatically triggering a sequence of actions that build, containerize, and deploy the application without downtime. This ensures faster and more reliable system updates, minimizing operational delays.

COURSE-END PROJECT

Industry Relevance

The following are the DevOps tools and AWS services used in this project:

1. AWS CodePipeline: A CI/CD service that automates the build, test, and deployment process of the application through a defined pipeline
2. AWS CodeBuild: A fully managed build service that compiles source code, runs tests, and produces software packages ready for deployment
3. AWS ECR (Elastic Container Registry): A fully managed Docker container registry that stores, manages, and deploys Docker container images
4. AWS ECS (Elastic Container Service): A scalable container orchestration service that enables you to run, stop, and manage containers on a cluster

COURSE-END PROJECT

Industry Relevance

The following are the DevOps tools and AWS services used in this project:

5. Docker: A platform for developing, shipping, and running applications inside lightweight, portable containers

6. IAM (Identity and Access Management): AWS service used to manage access to AWS resources, providing necessary permissions for CodeBuild and ECS

7. GitHub: A source code hosting platform that stores the Spring Boot application repository and integrates with the CI/CD pipeline.

COURSE-END PROJECT

Tasks

The following tasks outline the CI/CD pipeline creation process:

1. Fork the Spring Boot project GitHub repository and create an ECR to store the Docker images
2. Modify the build configuration file in the GitHub repository to include the ECR image URI
3. Create a CodeBuild project to pull code from the GitHub repository
4. Create an ECS cluster using the Fargate launch type and configure an ECS service to run tasks on the cluster
5. Deploy the application

COURSE-END PROJECT

Tasks

The following tasks outline the CI/CD pipeline creation process:

6. Configure CodePipeline to trigger on GitHub changes, integrating CodeBuild and ECS for automated deployment

7. Test and monitor the pipeline execution in AWS CodePipeline

COURSE-END PROJECT

Thank you